

COMP3211

03. Agile Software Development

Dr. Max Yu Pei

Intended Learning Outcomes

- ❖ Upon completion of this lecture, you will
 1. understand the rationale for agile software development methods, the agile manifesto, and the differences between agile and plan-driven development;
 2. know about important agile development practices such as user stories, refactoring, pair programming and test-driven development;
 3. understand the Scrum approach to agile project management;

Agile Development (or Agile Methods)

- ❖ Dissatisfaction with the overheads involved in plan-driven software development methods of the 1980s and 1990s led to the creation of agile methods.
 - The plan-driven approach was mainly used for developing specific types of software systems (e.g., large, long-lived software systems such as aerospace and government systems)
- ❖ The aim of agile development is to reduce overheads in the software process (e.g., by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.
 - All agile methods suggest that software should be developed and delivered incrementally.

Agile Manifesto (<http://agilemanifesto.org>)

- ❖ *We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:*

Individuals and interactions over processes and tools;

Working software over comprehensive documentation;

Customer collaboration over contract negotiation;

Responding to change over following a plan.

That is, while there is value in the items on the right, we value the items on the left more.

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is to provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

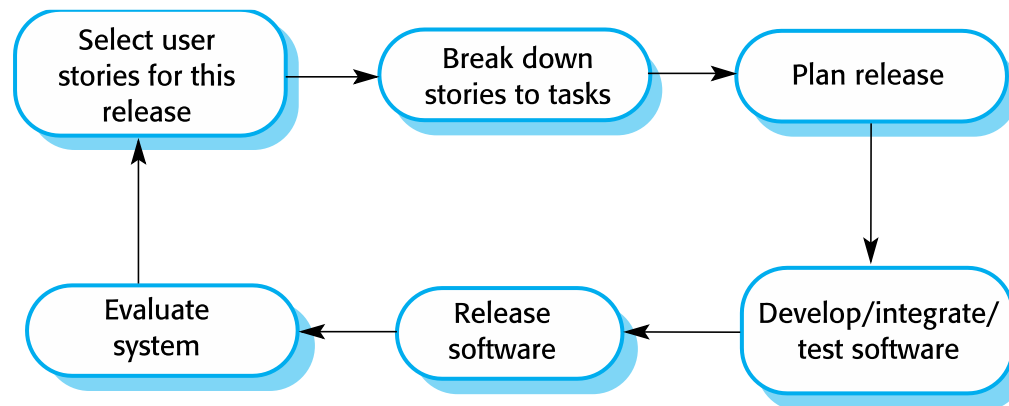
Agile Method Applicability

- ❖ Agile methods have been particularly successful for two kinds of system development.
 - Product development where a software company is developing a small or medium-sized product for sale.
 - Most software products and apps are now developed using an agile approach
 - Custom system development within an organization, where
 - 1) there is a clear commitment from the customer to become involved in the development process and
 - 2) there are few external rules and regulations that affect the software.
- Communication with customers and within the development team

AGILE DEVELOPMENT TECHNIQUES

Extreme Programming (XP)

- ❖ A very influential agile method, developed in the late 1990s, that introduced a range of agile development techniques.
- ❖ Pushing recognized good practice to “extreme” levels
 - New versions may be built several times per day;
 - Increments are delivered to customers every 2 weeks;
 - All tests must be run for every build and the build is only accepted if tests run successfully.
- ❖ The extreme programming release cycle



Influential XP Practices

- ❖ Extreme programming has a technical focus and is difficult to integrate with most organizations' management practices.
- ❖ Consequently, while agile development uses practices from XP, the method as originally defined is not widely used.
- ❖ Key practices
 - User stories for specification
 - Refactoring
 - Test-driven development
 - Pair programming

User Stories for Requirements (1)

- ❖ In XP, user requirements are expressed as user stories, i.e., informal, natural language descriptions of features of a software system
- ❖ User stories may follow one of several formats or templates.
 - As a <role> I can <capability>, so that <receive benefit>
 - As <who> <when> <where>, I want <what> because <why>
- ❖ Examples
 - As the HR manager, I want to create a screening quiz so that I can understand whether I want to send possible recruits to the functional manager.
 - As a user, I can indicate folders not to back up so that my backup drive is not filled up with things I do not need to be saved.

User Stories for Requirements (2)

- ❖ The three C's of user stories
 - Card: a written description of the story used for planning and estimation
 - Communication: discuss the idea with others
 - Confirmation: reach an agreement on what to build

- ❖ Advantages and disadvantages
 - ✓ People find it easier to relate to these stories than a conventional requirements document or use cases.
 - ✗ It is difficult to judge if enough user stories have been developed to cover all the essential requirements or if a single story gives a true picture of an activity.

“Design for Change” vs. Refactoring

- ❖ Conventional wisdom in software engineering is to design for change.
 - It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- ❖ XP proposes constant code improvement (refactoring) to make changes easier when they have to be implemented.
 - “Design for change” is not worthwhile as changes cannot be reliably anticipated.

Refactoring

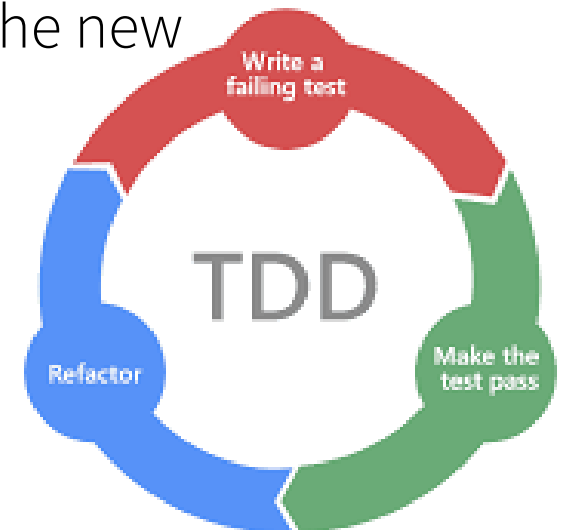
- ❖ Look for possible software improvements and make them even when there is no immediate need.
 - Tidying up and renaming attributes and methods to make them easier to understand.
 - The replacement of inline code with calls to methods included in a program library.
- ❖ Benefits and problems
 - This improves the understandability of the software and reduces the need for documentation. (Local changes tend to degrade software structure and readability)
 - Changes become easier to make because the code is well-structured and clear.
 - Refactoring sometimes gets delayed, and some changes require expensive architecture refactoring.

Testing in XP

- ❖ With XP, no system specification can be used by an external testing team to develop system tests
 - User stories are user requirements but not system requirements as they often miss important information about the system
- ❖ XP has developed an approach to test the program after every change. XP testing features:
 - Incremental test development from scenarios.
 - Test-driven development.
 - Customer involvement in test development and validation.
 - Use of automated testing frameworks

Test-Driven Development

- ❖ Tests are written as programs before code
 - Writing tests first avoids the problem of “test-lag”.
 - Writing tests implicitly defines both an interface and a specification of behavior for the functionality being developed
- ❖ Each test includes a check of whether its execution is correct.
- ❖ All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors.
 - Usually relies on a testing framework (such as Junit).



Problems with Test-Driven Development

- ❖ Programmers prefer programming to testing and sometimes they take short cuts when writing tests.
 - For example, they may write incomplete tests that do not check for all possible exceptions that may occur.
- ❖ Some tests can be very difficult to write incrementally.
 - For example, writing *End-to-End (E2E) Tests* for a complex, multi-step user journey *before* the individual components are built.
- ❖ It is difficult to judge the completeness of a set of tests.
 - Although you may have a lot of system tests, your test set may not provide complete coverage.

Customer Involvement

- ❖ The customer helps develop acceptance tests for the stories to be implemented in the next release of the system.

Test 4: Dose checking
Input: 1. A number in mg representing a single dose of the drug. 2. A number representing the number of single doses per day.
Tests: 1. Test for inputs where the single dose is correct but the frequency is too high. 2. Test for inputs where the single dose is too high and too low. 3. Test for inputs where the single dose * frequency is too high and too low. 4. Test for inputs where single dose * frequency is in the permitted range.
Output: OK or error message indicating that the dose is outside the safe range.

Test Case Description for Dose Checking

- ❖ Limitations
 - People adopting the customer role have limited time and cannot work full-time with the development team.
 - They may feel that providing the requirements was enough of a contribution and may be reluctant to get involved in the testing process.

Test Automation

- ❖ Test automation means that tests are written as executable components (before the task is implemented)
 - These testing components should be stand-alone, simulate the submission of input to be tested, and check that the result meets the output specification.
 - An automated test framework (e.g., JUnit) is a system that makes it easy to write executable tests and submit a set of tests for execution.
- ❖ As testing is automated, there is always a set of tests that can be quickly and easily executed
 - Whenever any functionality is added to the system, the tests can be run, and problems introduced by the new code can be caught immediately.

Pair Programming

- ❖ Programmers sit together at the same computer to develop the software.
 - Pairs are created dynamically so that all team members work with each other during the development process.
- ❖ Benefits
 - It supports the idea of collective ownership and responsibility for the system.
 - It acts as an informal review process because at least two people look at each line of code
 - It encourages refactoring to improve the software structure.
 - It is not necessarily inefficient.

XP Principles and Practices

Principle or practice	Description
Incremental planning	Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-driven development	A software development technique in which you write a failing automated test before the production code, then write just enough code to make it pass, and finally refactor while keeping all tests green.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.
Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium-term productivity
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team.

Support for Agile Principles in XP Practices

- ❖ Five agile principles (Slide 4)
 - Incremental delivery is supported through small releases.
 - Customer involvement is supported through on-site customers.
 - People, not process, are supported through pair programming, collective ownership, and sustainable pace.
 - Change is embraced through small releases, test-first development, refactoring, and continuous integration.
 - Maintaining simplicity is supported by simple design and refactoring.

AGILE PROJECT MANAGEMENT

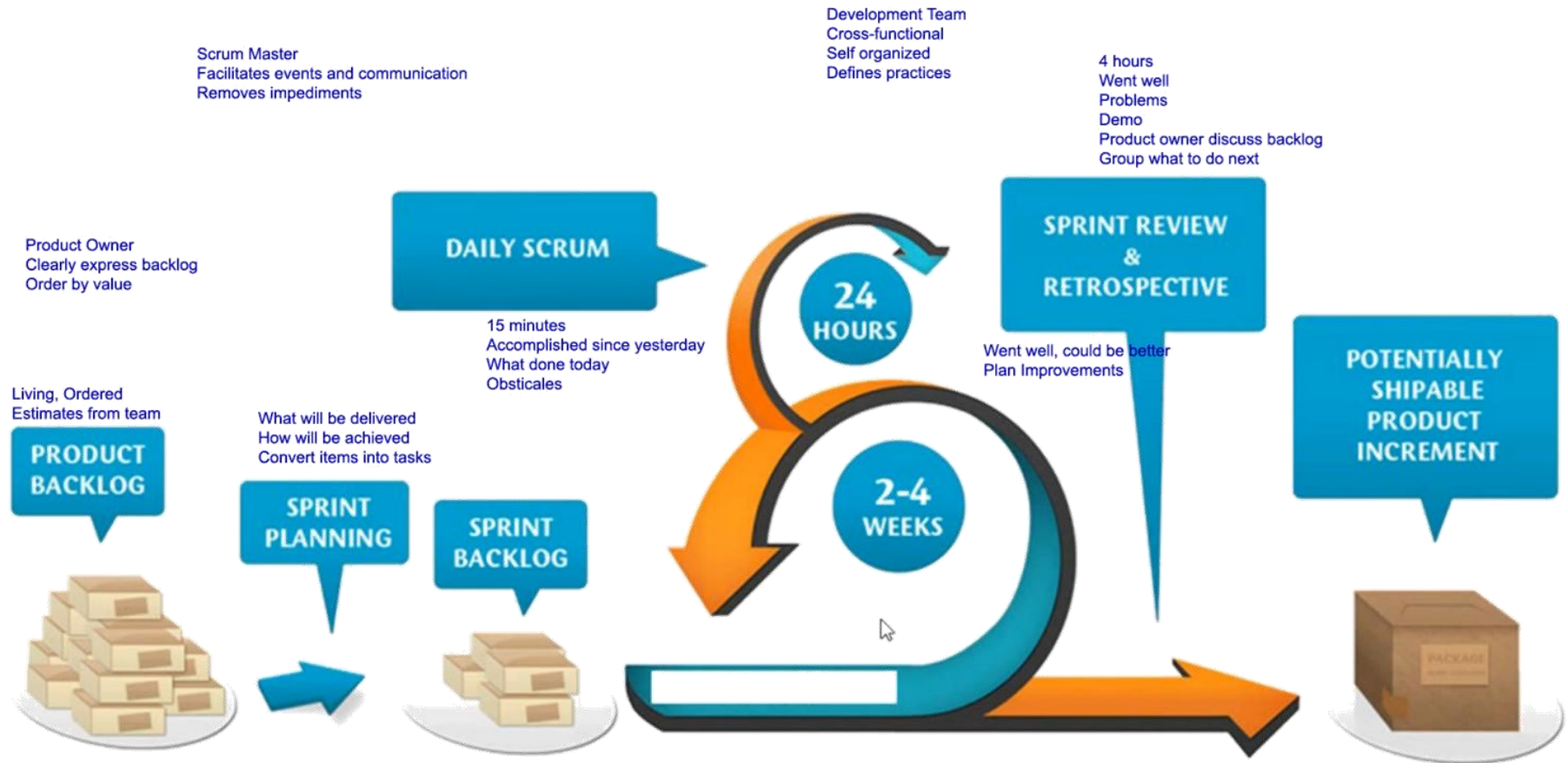
Agile Project Management

- ❖ Plan-driven software development requires a manager to have a stable view of everything that must be developed and the development processes.
- ❖ The informal planning and project control proposed by the early adherents of agile methods clashed with this business requirement for visibility.
 - Self-organizing teams
 - Little or no documentation
 - Short-term development plans
- ❖ Agile project management requires a different approach, adapted to incremental development and the practices used in agile methods.

Scrum

- ❖ Scrum is an agile method that focuses on managing iterative development rather than specific agile practices.
- ❖ There are three phases in Scrum.
 - The *initial* phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
 - That is followed by a series of *sprint cycles*, where each cycle develops an increment of the system.
 - The *project closure* phase wraps up the project, completes required documentation, and assesses the lessons learned.

Scrum Overview



The Scrum Sprint Cycle

- ❖ Sprints are fixed length, normally 2–4 weeks.
- ❖ The starting point for planning is the product backlog, which is the list of work on the project.
- ❖ The selection phase involves the whole project team who works with the customer to select the features and functionalities from the product backlog to be developed during the sprint.
- ❖ Once these are agreed on, the team organizes itself to develop the software.
- ❖ During this stage, the team is isolated from the customer and the organization, with all communications channelled through the so-called ‘ScrumMaster’.
- ❖ The role of the ScrumMaster is to protect the development team from external distractions.
- ❖ At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Teamwork in Scrum

- ❖ The ScrumMaster is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog, and communicates with customers and management outside of the team.
- ❖ The whole team attends short daily meetings (i.e., Scrums) where all team members share information and describe their progress since the last meeting, problems that have arisen, and what is planned for the following day.
 - This means that everyone on the team knows what is happening and, if problems arise, can re-plan short-term work to cope with them.

Scrum Terminology

Scrum Term	Definition
Development team	A self-organizing group of software developers, which should be no more than 7 people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be 'potentially shippable' which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of 'to do' items which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously review the product backlog to ensure that the project continues to meet critical business needs.
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference.
Sprint	A development iteration. Sprints are usually 2-4 weeks long.
Velocity	An estimate of how much product backlog effort that a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

Scrum Benefits

- ❖ The product is broken down into a set of manageable and understandable chunks that stakeholders can relate to.
- ❖ Unstable requirements do not hold up progress.
- ❖ The whole team has visibility of everything, and consequently, team communication is improved.
- ❖ Customers see on-time delivery of increments and gain feedback on how the product works.
- ❖ Trust between customers and developers is established, and a positive culture is created in which everyone expects the project to succeed.

Practical Problems with Agile Methods

- ❖ Agile methods may not be suitable for the development of large, long-lifetime systems for external clients
 - Contract, governance, and risk mismatch
 - Weak or fragmented product ownership
 - Architecture, scale, and integration challenges
 - Documentation, handover, and long-term maintenance

- ❖ They are most appropriate for new software development rather than software maintenance
 - Interrupt-driven work conflicts with sprint-based planning
 - Missing or weak product ownership and user access
 - Legacy code, regression risk, and architecture constraints
 - Governance, contracts, compliance, and long-term handover

Key Points

- ❖ Agile methods are incremental development methods that focus on rapid software development, frequent releases of the software, reducing process overheads by minimizing documentation, and producing high-quality code.
- ❖ Agile development practices include, e.g.,
 - user stories for system specification,
 - frequent releases of the software,
 - continuous software improvement,
 - test-driven development, and
 - customer participation in the development team.
- ❖ Scrum is an agile method that provides a project management framework. It is centered on a set of sprints, i.e., fixed time periods during which a system increment is developed.

THE END